

Resource-Aware Deep Learning

Tracking Energy Consumption in Scientific
AI Applications

Kevin Schmitz

 kevin2

Anno Knierim

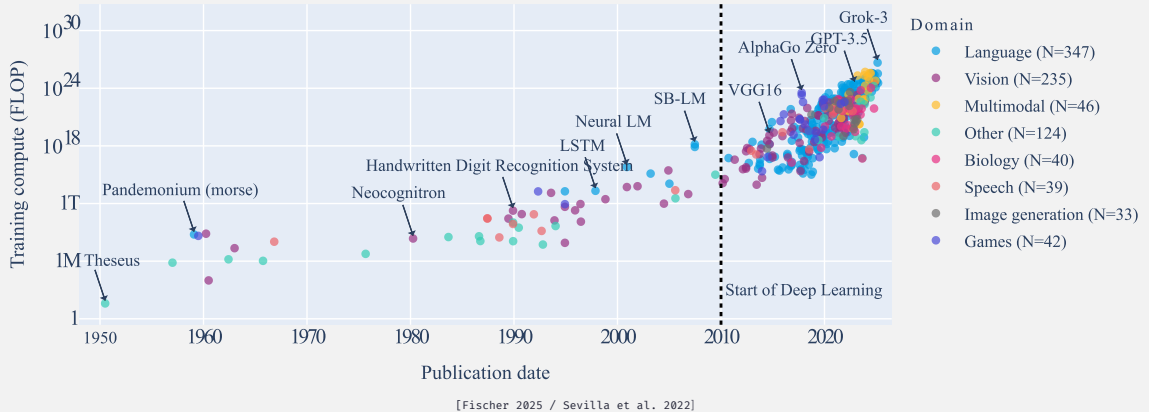
 aknierim

Raphael Fischer

 raphischer

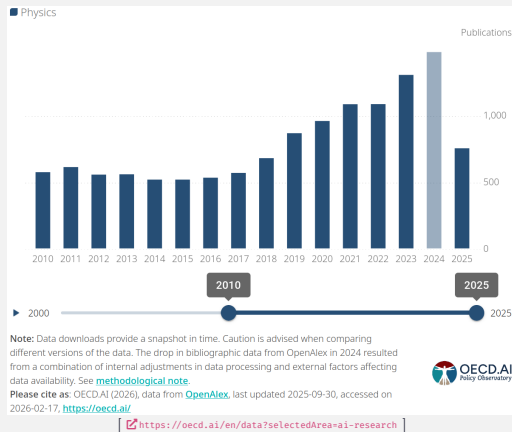
March 15, 2026

Why Resource Awareness Matters



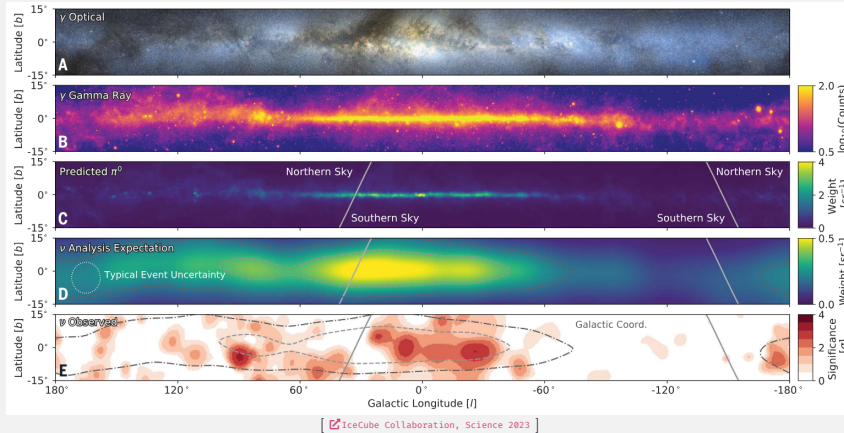
Why Resource Awareness Matters

- Growing computational demands of AI in physics
- Environmental impact: CO₂ footprint of training and inference
- Compute costs and computing cluster resource allocation
- Tracking resources as part of scientific documentation and reproducibility



AI Applications in Physics

- Source identification in astronomy
- Refinement in medical imaging
- Particle track reconstruction in high-energy physics
- Large language models for many applications



DFG Sustainability Guidelines



About Us ▾

Funding ▾

Basics and Topics ▾

Funded Projects ▾

News ▾

DFG > Basics and Topics > Developments within the Research System > **Sustainability Guide for Research Processes**

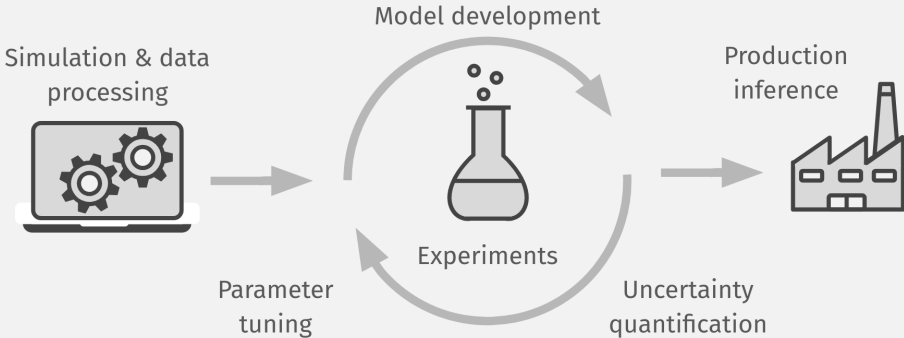


© DFG / AdobeStock

Sustainability Guide for Research Processes

[ DFG Sustainability Guide for Research Processes]

AI Workflows in Physics



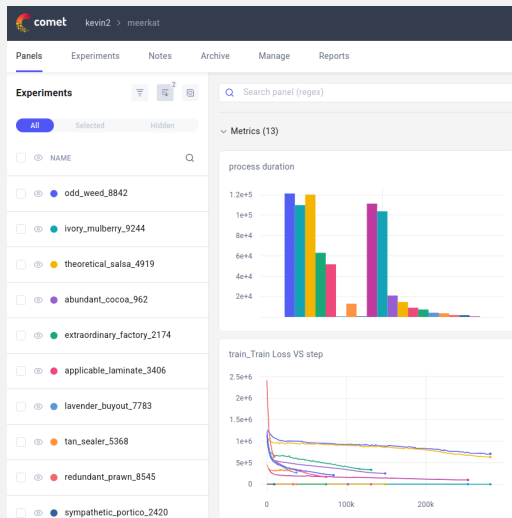
ML tracking/monitoring in scientific workflows

- Need for reproducibility in scientific research
- Documentation requirements for publications
- Integration with existing workflows
- MLflow^a / Weights & Biases^b / Comet^c

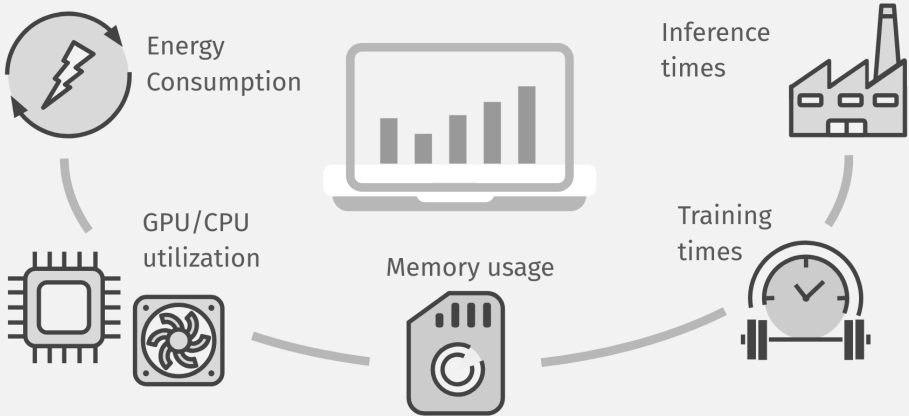
^amlflow.org

^bwandb.ai

^ccomet.com



Measuring Resource Consumption



Typical Resource Profile in Physics AI Applications

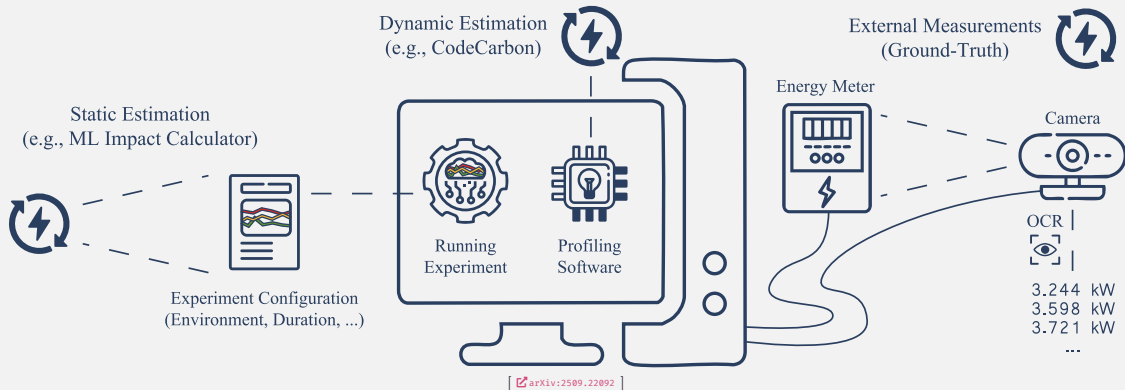
Workflow Stage	Time	Energy
Data Creation:		
Simulation Generation	X hours	Y kWh
Data Preprocessing	X hours	Y kWh
Development Cycle:		
Model Selection	X hours	Y kWh
Hyperparameter Tuning	X hours	Y kWh
Loss Selection	X hours	Y kWh
Production:		
Inference Run	X hours	Y kWh
Development Total	XX hours	YY kWh

Available Tracking Methods

Static Estimation

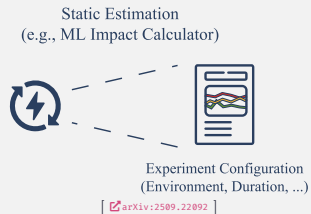
Dynamic Estimation

External Measurement



Static Estimation

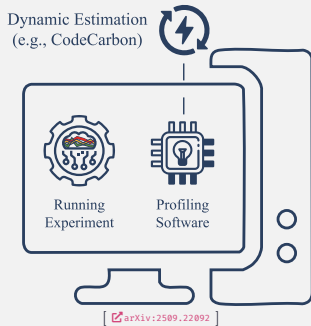
- Statistical estimation
- Based on configuration and environment
- $\text{CO}_2 \text{ Equiv.} = \text{Power} \cdot \text{Time} \cdot \text{CO}_2 \text{ Efficiency}$
- ML Impact Calculator^a
- No flexible adjustments



^a climatechange.ai/papers/neurips2019/22

Dynamic Estimation

- Profiling dynamic resource demand
- Power x Running Time
- Profiling software: experiment-impact-tracker^a / CodeCarbon^b
- Power Draw per short time interval
- Misses power supply and cooling

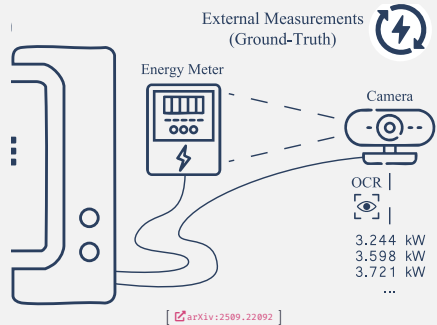


^a[arxiv:2002.05651](https://arxiv.org/abs/2002.05651)

^bcodecarbon.io/

External Measurements

- Ground-truth energy consumption
- Capture impact of all components
→ external measurement
- Energy meter + camera + computer vision code
- Energy meter between power supply and hardware
- But: additional hardware and implementation effort



Combined Tracking

Static Estimation

Dynamic Estimation

External Measurement



[arXiv:2509.22092](https://arxiv.org/abs/2509.22092)

CodeCarbon: Overview

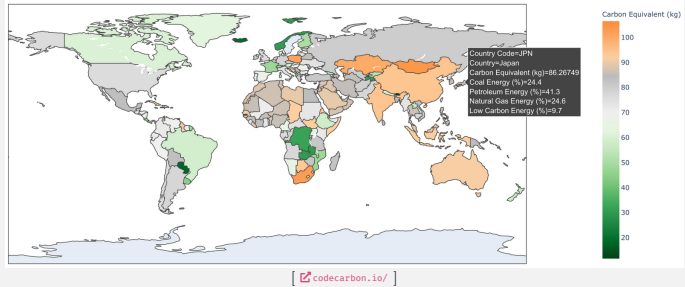
- CO₂ emission tracking
- Developed by open-source community
- CPU, GPU and TPU workloads
- Real-time monitoring and post-run analysis
- Multi-GPU support
- < 1% performance overhead



[codecarbon.io/]

CodeCarbon: Data Output

- Energy consumption (kWh)
- CO₂ emissions (kg CO₂eq)
- Duration
- Timestamp
- Hardware details (CPU/GPU/TPU type, count)
- Regional CO₂ factors



CodeCarbon: Getting Started

- GitHub: [mlco2/codecarbon](https://github.com/mlco2/codecarbon)
- Installable via pip
- CSV output with emissions data
- Automatic logging (e.g. CometML/W&B)

Python

```
$ uv pip install codecarbon
```

```
from codecarbon import EmissionsTracker
```

```
tracker = EmissionsTracker()  
tracker.start()
```

```
# Your training code here  
# ...
```

```
# Stop tracking  
emissions = tracker.stop()
```

CodeCarbon: Practical Considerations

Good for:

- Comparing model architectures
- Hyperparameter optimization experiments
- Single-machine or single-node workloads
- Quick integration into existing pipelines

Struggles with:

- Ground-truth validation
- Multi-node HPC jobs
- Accuracy: $\pm 10 - 20\%$ for GPU workloads
- Average regional CO₂ factors

LAMARR Energy Tracker (LET)

- Installable via pip
\$ `uv pip install lamarr-energy-tracker`
- Wrapper around CodeCarbon
- Prints energy consumption as well as a summary statement
- Stores emission data centrally in `~/.let/emissions.csv` (default)
- Comes with CLI functionality



 [lamarr-institute/LET](https://github.com/lamarr-institute/LET)

LAMARR Energy Tracker (LET)

```
from lamarr_energy_tracker import EnergyTracker

# Either use as a context manager
with EnergyTracker(project_name="your_research_project") as tracker:
    # Your resource-heavy code here
    pass

# Or manually
tracker = EnergyTracker(project_name="your_research_project")
tracker.start()
# Your resource-heavy code here
tracker.stop()
```

LAMARR Energy Tracker (LET)

```
$ python your_tracked_code.py
```

```
...
```

Using CodeCarbon 3.0.8, the energy consumption of running all experiments on an Intel(R) Core(TM) i7-10610U CPU is estimated to 0.135 kWh. This corresponds to estimated carbon emissions of 0.051 kg of CO₂-equivalents, assuming a carbon intensity of 380 gCO₂/kWh~\cite{lamarr_energy_tracker,codecarbon}. Note that these numbers are underestimations of actual resource consumption and do not account for overhead factors or embodied impacts~\cite{ai_energy_validation}.

Example: Logging with MLFlow and LET

```
import mlflow
import pandas as pd
from lamarr_energy_tracker import EnergyTracker
```

Example: Logging with MLFlow and LET

```
import mlflow
import pandas as pd
from lamarr_energy_tracker import EnergyTracker

with EnergyTracker(project_name="deep_learning", output_dir="/model/save/path") as tracker:
    # Your training/inference here
    mlflow.log_metric(...) # e.g., loss
    mlflow.log_param(...) # e.g., metadata (trainable params, batch size, architecture)
    pass
```

Example: Logging with MLFlow and LET

```
import mlflow
import pandas as pd
from lamarr_energy_tracker import EnergyTracker

with EnergyTracker(project_name="deep_learning", output_dir="/model/save/path") as tracker:
    # Your training/inference here
    mlflow.log_metric(...) # e.g., loss
    mlflow.log_param(...) # e.g., metadata (trainable params, batch size, architecture)
    pass

# After training/inference
emission_data = pd.read_csv("/model/save/path/emissions.csv").to_dict()
```


Example: Logging with MLFlow and LET

```
import mlflow
import pandas as pd
from lamarr_energy_tracker import EnergyTracker

with EnergyTracker(project_name="deep_learning", output_dir="/model/save/path") as tracker:
    # Your training/inference here
    mlflow.log_metric(...) # e.g., loss
    mlflow.log_param(...) # e.g., metadata (trainable params, batch size, architecture)
    pass

# After training/inference
emission_data = pd.read_csv("/model/save/path/emissions.csv").to_dict()

eval_res = dict(
    running_time_total=emission_data["duration"][0],
    power_draw_total=emission_data["energy_consumed"][0] * 3.6e6,
    ...
)

for key, val in eval_res.items():
    mlflow.log_metric(key=key, value=value)
```

Example: Logging with MLFlow and LET – Now what?

Export the logged data to a csv file.

```
$ MLFLOW_TRACKING_URI="file:./path/to/mlruns/" mlflow experiments csv \  
-x <experiment_id> -o export.csv
```

Example: Logging with MLFlow and LET – Now what?

Export the logged data to a csv file.

```
$ MLFLOW_TRACKING_URI="file:./path/to/mlruns/" mlflow experiments csv \  
-x <experiment_id> -o export.csv
```

We can now visualise our results using STREP.

STREP: Sustainable and Trustworthy Reporting for ML and AI

- Available on PyPI:
`$ uv pip install strep[frontend]`
- Visualise your data with a single script and a JSON file for metadata

```
from strep.index_scale import load_database,  
    ↪ scale_and_rate  
from strep.elex.app import Visualization  
  
fname = 'path/to/your/database.pkl' # or .csv summary  
# load database and meta information (if available)  
database, meta = load_database(fname)  
# index-scale and rate database  
rated_database = scale_and_rate(database, meta)  
# start the interactive exploration tool  
app = Visualization(rated_database)  
app.run()
```

```
meta_properties.json  
{  
    "num_trainable_parameters": {  
        "name": "Parameters",  
        "shortname": "PAR",  
        "unit": "Number",  
        "group": "Resources"  
    },  
    "power_draw": {  
        "name": "Power Draw per  
        ↪ Epoch",  
        "shortname": "ENE",  
        "unit": "Wattseconds",  
        "group": "Resources"  
    }  
}
```

Final Advices



Iterative Integration

- Add CodeCarbon
- Integrate STREP for shared monitoring
- Ground-truth validation



Common Routines

- Track by default
- Native optimization
- Model performance vs energy



Detailed Documentation

- ML tracking platforms
- Model cards with carbon footprint
- Used software and hardware details



Data Sharing

- Code on GitHub
- Trained models on HuggingFace
- Datasets on Zenodo/HuggingFace

Hands-on Session

Try emission tracking yourself:

 [radionets-project/resource_awareness](https://github.com/radionets-project/resource_awareness)

